

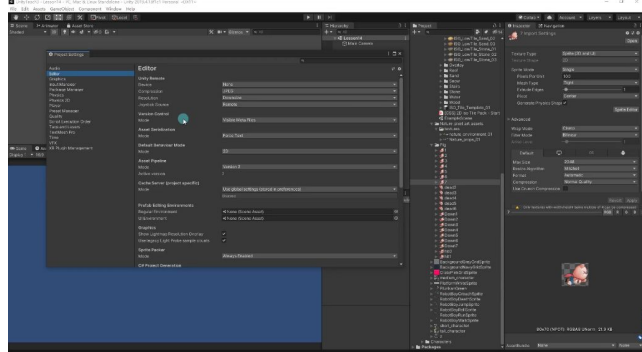
一、精灵图集制作 00:04

1. 为什么要打图集 00:24

- **核心目的**：减少DrawCall数量，提高游戏性能
- **应用场景**：在2D游戏开发和UI开发中会频繁使用
- **性能影响**：当DrawCall数量较多时，游戏可能会出现卡顿现象
- **实现效果**：通过将多张小图合并成大图集，减少渲染批次，使游戏运行更流畅

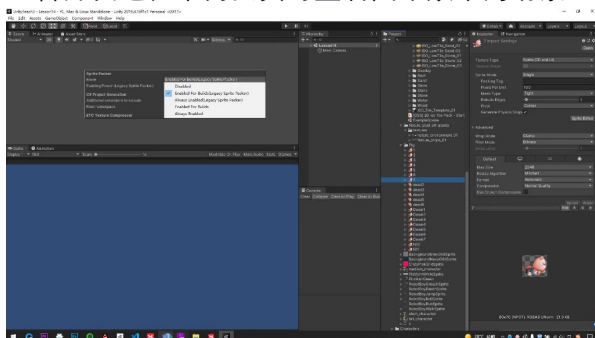
1) 在Unity中打开自带的打图集功能 01:26

- **功能入口** 01:48
 - **路径**：Edit->Project Setting->Editor
 - **选项名称**：Sprite Packer(精灵包装器)
 - **默认状态**：非2D项目默认Disabled，2D项目默认Always Enabled



● 不同模式的区别 03:44

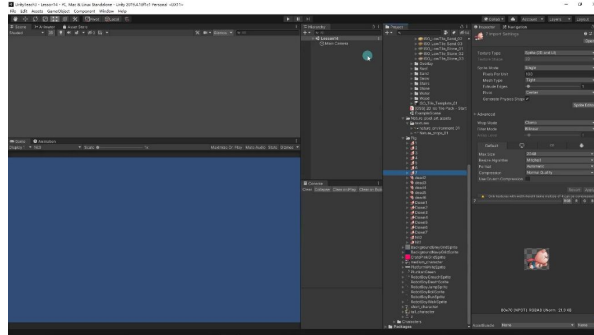
- **Disabled**:
 - 默认设置，不会打包图集
 - 适用于非2D项目或不需要图集功能的场景
- **Enabled For Builds(Legacy Sprite Packer)**:
 - Unity仅在构建时打包图集
 - 编辑器模式下不会打包
 - 适用于发布前才需要打包的场景
- **Always Enabled(Legacy Sprite Packer)**:
 - Unity在构建时和编辑器运行前都会打包图集
 - 适合开发过程中需要实时查看图集效果的场景



● Padding Power参数解释 05:27

- **作用**：设置图片之间的间隔距离
- **计算方式**：数值代表2的n次方像素
 - $1 = 2^0$ 像素
 - $2 = 2^1$ 像素

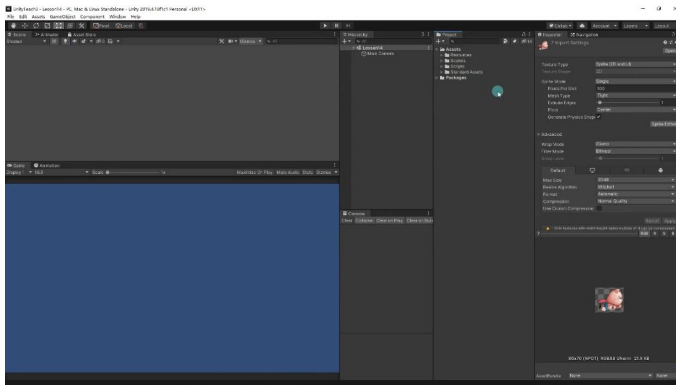
- 3 = 8像素(2^3)
 - 应用范围：仅Legacy Sprite Packer模式下可用
- 推荐使用的模式 06:39
 - 最佳实践：选择Always Enabled模式
 - 优势：
 - 开发过程中可实时查看图集效果
 - 避免构建时才发现图集问题
 - 2D项目默认即为该模式



- 注意事项：
 - 非2D项目需要手动开启该功能
 - 传统打包模式(Legacy)多了一个间隔距离设置参数
 - 新项目建议直接使用2D模板以获得最佳默认配置

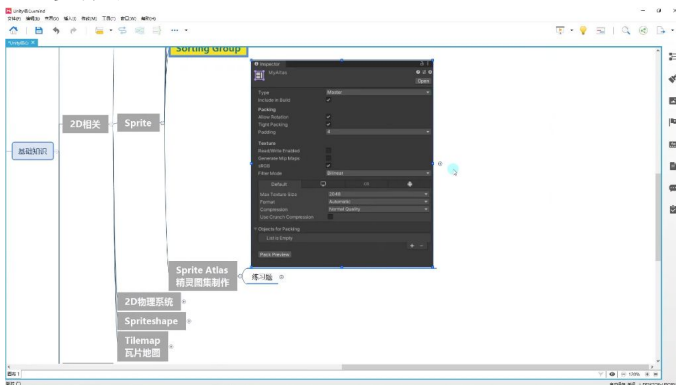
2. 打图集面板参数相关 07:14

1) 创建图集对象 07:36



- 创建方法：在Project窗口右键→Create→Sprite Atlas
- 存储位置：建议创建在Resources文件夹下以便动态加载
- 命名规范：可自定义名称如"MyAtlas"

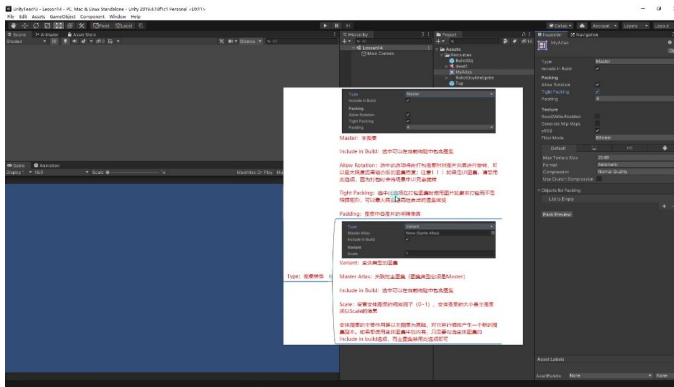
2) 图集参数设置 08:15



- 参数分类：包含打包参数和纹理参数两大类

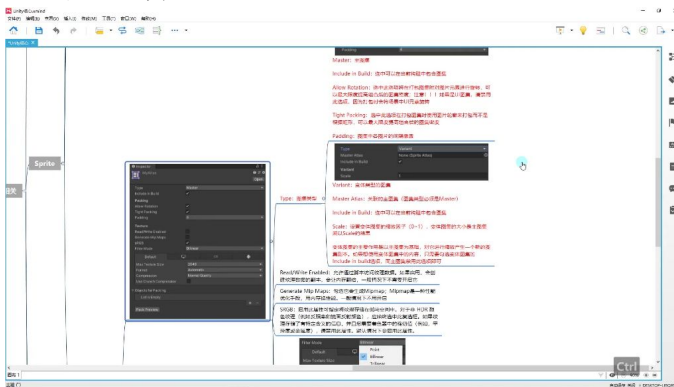
- 相似性说明：部分参数与图片导入设置类似，主要关注新增参数

3) 主图集与变体图集 09:01



- **Master类型**：主图集，最常用的图集类型
- **Variant类型**：变体图集，基于主图集缩放产生
- **使用频率**：主图集使用率90%以上，变体图集仅特殊情况下使用

4) 图片打包选项 09:36

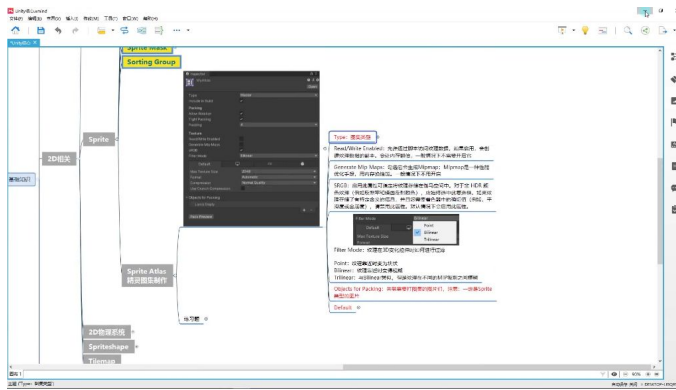


- **Allow Rotation**：
 - 作用：允许旋转图片元素以提高图集密度
 - 注意：UI图集必须禁用此选项
 - 原理：自动旋转长条形图片以节省空间
- **Tight Packing**：
 - 作用：根据图片轮廓而非矩形边界打包
 - 优势：可减少透明区域占用空间
 - 效果：整体减小图集尺寸约20-30%

5) 图集间隔像素 12:47

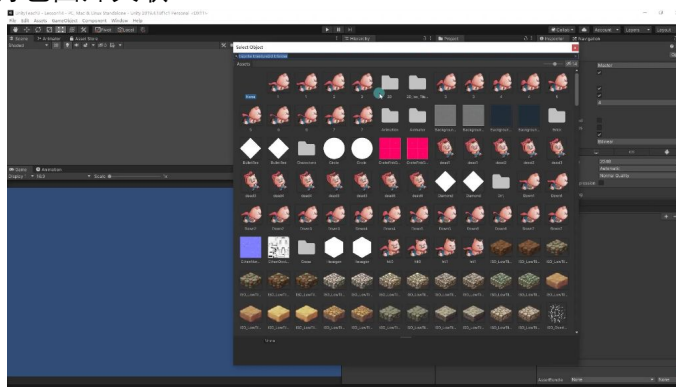
- **Padding设置**：
 - 含义：图片间的间隔距离
 - 数值规则：必须为 2^n (2/4/8/16等)
 - 典型值：4或8像素

6) 纹理相关参数 14:51



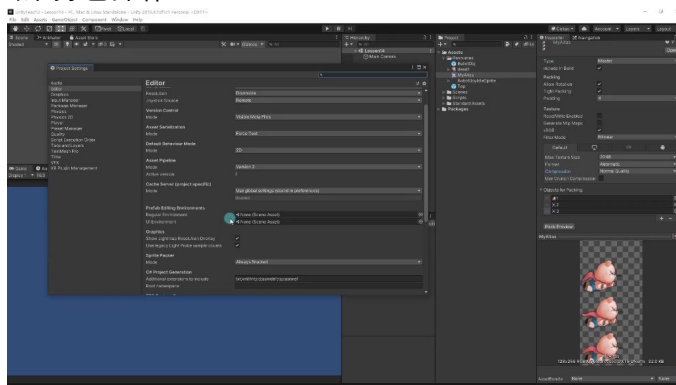
- **Read/Write Enabled:**
 - 作用：允许脚本访问纹理数据
 - 内存影响：启用会使内存翻倍
 - 建议：非必要不启用
- **Generate Mip Maps:**
 - 作用：生成多级纹理优化性能
 - 适用性：2D游戏通常不需要
- **sRGB:**
 - 作用：启用伽马空间校正
 - 默认值：建议保持启用

7) 打包图片关联 17:02



- 添加方式：
 - 通过Objects for Packing区域的"+"按钮
 - 直接拖拽图片或文件夹到该区域
 - 批量添加：推荐将同类图片放入文件夹后整体拖入
- 8) 图集打包操作 17:32

8) 图集打包操作 17:32

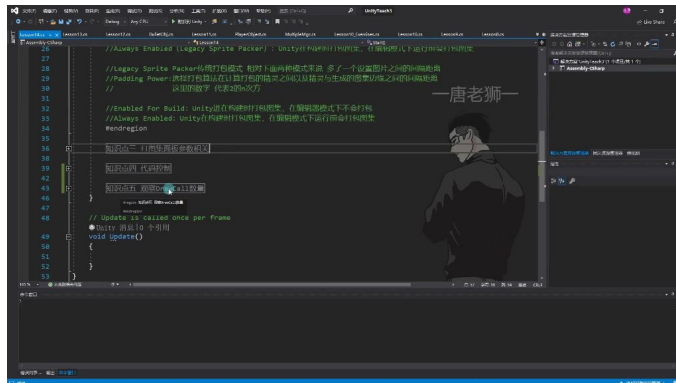


- 打包前提：需在Editor→Project Settings→Editor中启用Sprite Packer
- 打包验证：打包后可在Inspector窗口查看生成的图集

9) 快捷打包方式 18:48

- **拖拽操作：**
 - 单图拖拽：直接拖到Objects for Packing区域
 - 文件夹拖拽：自动包含文件夹内所有图片
- **美术规范：**建议按功能模块组织图片文件夹结构

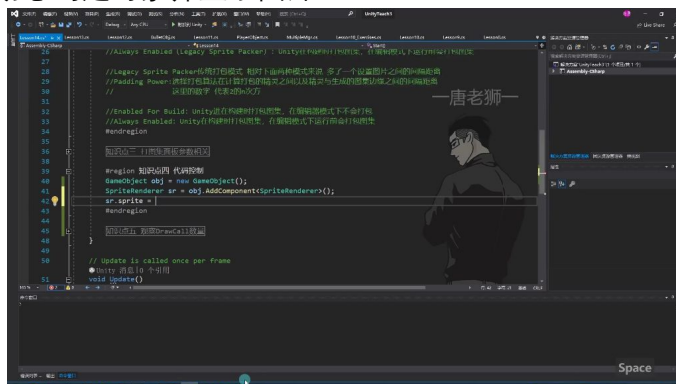
10) 图集使用与验证 20:28



- **使用方式：**正常使用原图片即可，Unity自动识别图集
- **效果验证：**
 - 查看Game窗口的Stats面板
 - 对比Batches数值变化
 - 图集生效后相同图集的图片共享一个DrawCall
- **性能对比：**
 - 未打包：每张图片独立DrawCall
 - 已打包：同图集图片共享1个DrawCall

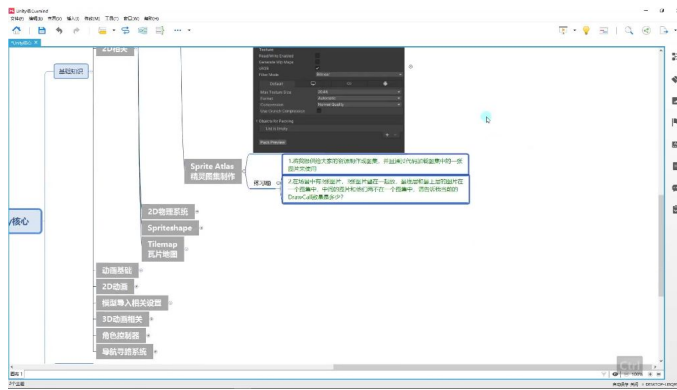
3. 代码控制 25:03

1) 动态创建对象并显示图片 25:21



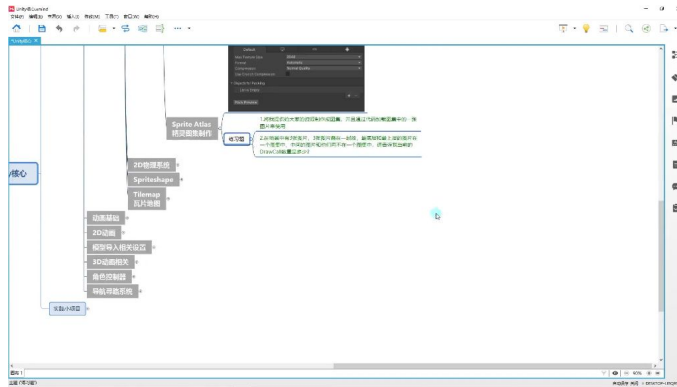
- **创建流程：**
 - 通过GameObject obj = new GameObject()动态创建游戏对象
 - 使用AddComponent<SpriteRenderer>()添加精灵渲染组件
 - 通过sprite属性设置要显示的图片

2) 加载图集资源 26:43



- 练习目标：
 - 将提供的资源制作成图集
 - 通过代码加载并使用图集集中的图片
- 关键点：
 - 掌握图集制作全流程
 - 熟练使用SpriteAtlas类API

2) 练习题：DrawCall分析



- 场景设置：
 - 3张叠放的图片
 - 底层和顶层使用同一图集
 - 中层使用不同图集
- 考察重点：
 - 理解图集对DrawCall的影响机制
 - 掌握性能优化原理

二、知识小结

知识点	核心内容	易混淆点	难度系数
为什么要打图集	减少DrawCall 提高性能，在2D游戏和UI开发中频繁使用	与外部预切割图集概念区分	★★
Unity打图集功能开启	需在Project Settings > Editor > Sprite Packer中启用，2D项目默认开启	构建时打包与编辑模式打包模式区别	★★★

图集面板参数	Master主图集类型、允许旋转优化密度、轮廓打包、像素间隔设置	UI图集必须禁用旋转选项	★★★★
代码控制图集	通过SpriteAtlas类加载，使用GetSprite()方法获取子图	Resources加载路径规范	★★★
性能验证方法	通过Game视图Stats面板观察Batches数值变化验证图集效果	运行前后批处理次数对比	★★
变体图集	基于主图集缩放生成，通过Scale因子控制大小	需关联主图集且主图集禁用Include in Build	★★★★
纹理参数设置	Read/Write禁用节省内存、sRGB保持勾选、2D项目禁用MipMaps	压缩格式与最大尺寸设置	★★★